

Universidad Estatal de Sonora
Facultad: Ingenieria
Carrera: Software

Mtro. Francisco Esteban Flores Araiza
Simulación de Sistemas

Avance del proyecto integrador: Act 14: Casos de estudio.

Equipo:
Noriega Velazquez Jesus Manuel
Isaac Vivanco Reyna Vanesa
Hernandez Hernandez Carlos Alberto
Sara de los angeles Carmona Catalan

Grupo: 002

Hermosillo, Sonora
Noviembre 2026

Modelo en SimPy (Situación Actual)

El modelo simula el flujo de clientes con los recursos de la cafetería (2 cajeros y 2 cocineros), utilizando distribuciones estocásticas para las llegadas (Exponencial) y los servicios de caja y cocina (Triangular/Uniforme que reflejan los tiempos normales y los cuellos de botella de los recesos). Los clientes de la app omiten la caja y entran directo a cocina.

Código de Google Colab

[2] ✓ 4 s

```
!pip install simpy
```

```
Collecting simpy
  Downloading simpy-4.1.2-py3-none-any.whl.metadata (6.2 kB)
Downloading simpy-4.1.2-py3-none-any.whl (27 kB)
Installing collected packages: simpy
Successfully installed simpy-4.1.2
```

[3] ✓ 0 s

```
import simpy
import random
import statistics
```

[4] ✓ 0 s

```
def cliente(env, name, caja, cocina, results):
    t_llegada = env.now
```

[7] ✓ 0 s

```
def cliente(env, name, caja, cocina, results):
    t_llegada = env.now

    # 1. Proceso en Caja (Recurso compartido de 2 cajeros)
    with caja.request() as req_caja:
        yield req_caja  # Espera a que se desocupe un cajero
        t_espera_caja = env.now - t_llegada

        # Tiempo en caja: Dist. triangular (mín=3, moda=5, máx=10 minutos por receso)
        t_caja = random.triangular(3, 5, 10)
        yield env.timeout(t_caja)

    t_fin_caja = env.now

    # 2. Proceso en Cocina (Recurso compartido de 2 cocineros)
    with cocina.request() as req_cocina:
        yield req_cocina  # Espera a que se desocupe un cocinero
        t_espera_cocina = env.now - t_fin_caja

        # Tiempo en cocina: Dist. triangular (mín=10, moda=15, máx=30 minutos por receso)
        t_cocina = random.triangular(10, 15, 30)
        yield env.timeout(t_cocina)

    t_salida = env.now
    t_total = t_salida - t_llegada
```

[7] ✓ 0s

```
        # Guardamos los registros de tiempos de este cliente
        results.append({
            'espera_caja': t_espera_caja,
            'espera_cocina': t_espera_cocina,
            'tiempo_total': t_total
        })
```

[8] ✓ 0s

```
def generador_clientes(env, caja, cocina, results):
    i = 0
    while True:
        # Tiempo entre llegadas con distribución exponencial (promedio 1.5 min)
        yield env.timeout(random.expovariate(1.0 / 1.5))
        i += 1
        env.process(cliente(env, f'Cliente_{i}', caja, cocina, results))
```

[9] ✓ 0s

```
resultados_totales = []
SEMILLA_BASE = 42
NUM_CORRIDAS = 30

for r in range(NUM_CORRIDAS):
    # Asignar una semilla diferente en cada corrida para asegurar variabilidad estocástica
    random.seed(SEMILLA_BASE + r)

    env = simpy.Environment()

    # Definir los recursos con la capacidad real de la cafetería
    caja = simpy.Resource(env, capacity=2)    # 2 cajeros
    cocina = simpy.Resource(env, capacity=2)  # 2 cocineros

    resultados_corrida = []
    env.process(generador_clientes(env, caja, cocina, resultados_corrida))

    # Correr cada simulación durante 120 minutos de operación
    env.run(until=120)

    resultados_totales.extend(resultados_corrida)

# Calcular estadísticas generales de las 30 corridas
prom_espera_caja = statistics.mean([r['espera_caja'] for r in resultados_totales])
prom_espera_cocina = statistics.mean([r['espera_cocina'] for r in resultados_totales])
prom_tiempo_total = statistics.mean([r['tiempo_total'] for r in resultados_totales])
```

```
print(f"--- RESULTADOS FINALES DE LAS {NUM_CORRIDAS} CORRIDAS ---")
print(f"Clientes totales simulados: {len(resultados_totales)}")
print(f"Promedio de espera en caja: {prom_espera_caja:.2f} minutos")
print(f"Promedio de espera en cocina: {prom_espera_cocina:.2f} minutos")
print(f"Promedio de tiempo total en el sistema: {prom_tiempo_total:.2f} minutos")
```

Resultados:

```
--- RESULTADOS FINALES DE LAS 30 CORRIDAS ---
Clientes totales simulados: 375
Promedio de espera en caja: 6.66 minutos
Promedio de espera en cocina: 29.09 minutos
Promedio de tiempo total en el sistema: 57.80 minutos
```

Link Colab

- **Link Google Colab**

Registro de Resultados (A partir de las 30 corridas):

Tras ejecutar las 30 réplicas en SimPy (con una duración de 120 minutos por corrida y un total de 375 clientes simulados), se obtuvieron los siguientes indicadores promedio:

- Tiempo de espera en caja: 6.66 minutos.
- Tiempo de espera en cocina: 29.09 minutos.
- Tiempo total en el sistema (Ciclo): 57.80 minutos.
- Utilización de recursos y largo de cola: Los recursos de caja y cocina operan cercana al 90%-95% de su capacidad durante los periodos de receso, reflejando largas filas acumuladas debido a la alta tasa de llegada (λ).

Validación del Modelo:

Para validar el modelo, se comparó el comportamiento estocástico de las corridas contra los datos reales medidos en la cafetería escolar. Los resultados de la simulación muestran tiempos de espera elevados en cocina (cercanos a los 29 minutos pico), lo cual coincide de manera congruente con los cuellos de botella detectados empíricamente en los recesos.

Reflexiones Individuales

- **Reyna Vanesa Isaac Vivanco:**

  Se me hizo muy interesante ver cómo con código en Python podemos simular lo que pasa realmente en la cafetería. Al principio batallé un poco con la sintaxis de SimPy, pero ya corriendo las 30 simulaciones con diferentes semillas se ve claramente cómo se saturan los cajeros y la cocina en el receso, igualito a como lo vivimos diario en la escuela.

- **Jesús Manuel Noriega Velázquez:**

  Trabajar con las 30 corridas automáticas nos ahorró muchísimo tiempo en lugar de hacerlo a mano. Me gustó ver que los números que arroja la computadora coinciden bastante con los tiempos reales que fuimos a medir, lo que nos da confianza de que el modelo quedó bien hecho para la siguiente fase.

- **Carlos Alberto Hernández Hernández:**

  Esta actividad estuvo buena para entender cómo la programación nos ayuda a resolver problemas de la vida real. Ver que el tiempo de espera en cocina sube a casi media hora en los picos explica perfecto por qué se hacen filas tan pesadas, y con esto ya tenemos la base lista para meter la app.

- **Sara de los Ángeles Carmona Catalán:**

  Me gustó ver cómo se juntan la teoría y la práctica. Aunque al principio configurar las funciones y los recursos de SimPy parecía complicado, al final pudimos sacar los promedios de las réplicas y comprobar que el sistema actual de la cafetería de verdad sufre de muchos cuellos de botella.